

计算机程序设计基础(2)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所
罗姆楼6-104

电话: 13901216180 / 62796193

2023.2.

1

教学目标

- 本课主要是培养掌握面向对象的程序设计基本原理、概念、方法和编程解题的思路与典型方法，达到可分析和设计综合程序的能力。
- 掌握类的基本概念、类的封装、类的继承性和多态性、函数重载与运算符重载等C++语言的基本知识。
- 并进一步掌握VS编程环境下的C++程序开发和调试方法。

2

教材和参考书

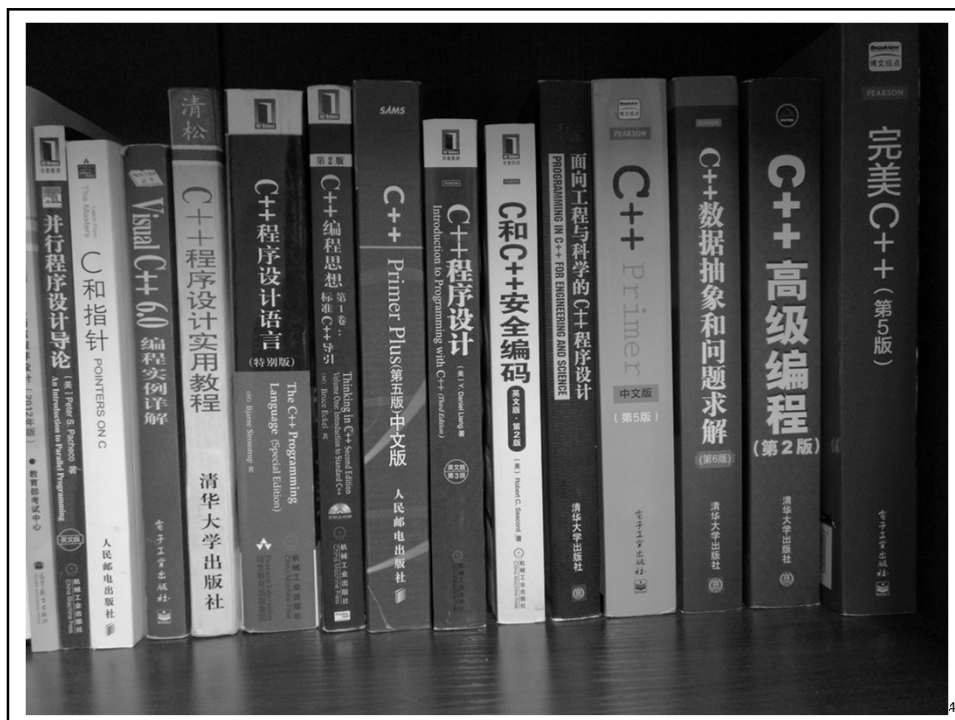
1. 教材

黄永峰, 孙甲松: 《C/C++程序设计教程》下篇
清华大学出版社 2019年6月第1版

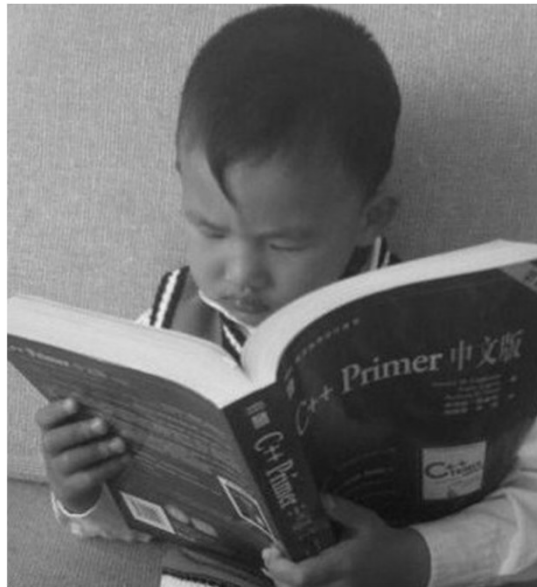
2. 主要参考书

- (1) 郑莉: 《C++语言程序设计(第4版)》清华大学出版社
- (2) Stephen Prata: 《C++ Primer Plus 中文版》第5版,
人民邮电出版社
- (3) Bruce Eckel: 《C++编程思想(Thinking in C++, 2nd/plus ed)》
中文版, 机械工业出版社
- (4) Bjarne Stroustrup: 《C++程序设计语言 (特别版)》中文版,
机械工业出版社

3



4



5

➤ 教学学时

C++课堂讲授24学时，授课12次，第13周或第14周结束C++授课，第15周周末闭卷笔试。

➤ 考核方式及成绩评定

最终提交的成绩中，期末笔试占40%，平日作业占30%，夏季学期大作业占15%，夏季学期Python作业及大作业占15%。本课没有机考。

➤ 平日作业

平时作业上机完成，一共11次，从第1周开始，每次2-3个题目，第2周开始上机。按二级选课五个时间段（周一下午，周二晚上，周三晚上，周四上午，周四下午）上机，题目和要求参见相应课件。

6

➤ 夏季学期C++大作业

夏季学期C++大作业以实验为主，第1周每人每天4个机时，共20个机时。完成1个大作业题目，并按照样例写实验报告。

➤ 夏季学期Python作业及大作业

夏季学期第2周学习Python语言，有三次Python语言讲座，完成三次Python作业与大作业。

夏季学期实验时间安排和具体要求到本学期末另行通知。

7

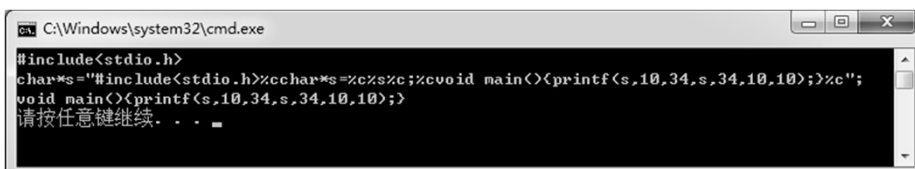
引言

有程序：

```
#include<stdio.h>
char*s="#include<stdio.h>%cchar*s=%c%c%c;%cvoid main(){printf(s,10,34,s,34,10,10);}%c";
void main(){printf(s,10,34,s,34,10,10);}
```

运行的结果是：

```
#include<stdio.h>
char*s="#include<stdio.h>%cchar*s=%c%c%c;%cvoid main(){printf(s,10,34,s,34,10,10);}%c";
void main(){printf(s,10,34,s,34,10,10);}
```



程序输出的结果和程序的源代码完全一样！

8

有程序：

```
main(){char *a;printf(a,34,a="main(){char *a;printf(a,34,a=%c%s%c,34);}",34);}
```

因为程序开头没有`#include <stdio.h>`，此程序编译时有警告错误信息，但可以正常运行，运行的结果是：

```
main(){char *a;printf(a,34,a="main(){char *a;printf(a,34,a=%c%s%c,34);}",34);}
```

程序运行的结果和程序的源代码也完全一样！

C语言功能强大，希望大家进一步巩固所学过的C语言内容，筑牢下一步学习C++的基础。

注意：

C程序文件名后缀： .c

C++程序文件名后缀： .cpp

9

说明一下为什么刚才第二个程序输出是程序本身：

```
main()
```

```
{char *a;
```

```
    printf(a,34,a="main(char*a){printf(a,34,a=%c%s%c,34);}",34);
```

```
}
```

34是字符“”的ASCII值，函数参数传递顺序自右至左，因此输出语句：

```
printf(a,34,a="main(char*a){printf(a,34,a=%c%s%c,34);}",34);
```

等价于：

```
printf("main(){char*a;printf(a,34,a=%c%s%c,34);}",
```

```
    34, "main(){char*a;printf(a,34,a=%c%s%c,34);}", 34);
```

第1个%c输出字符“”， %s输出后面双引号中的字符串：

```
    main(){char*a;printf(a,34,a=%c%s%c,34);}
```

第2个%c输出字符“”

因此整个printf函数语句输出的是：

```
main(){char*a;printf(a,34,a="main(){char*a;printf(a,34,a=%c%s%c,34);}",34);}
```

10

如果你看到如下的源程序代码：

```
x = a.Get(a);  
a.Put(&a, x+1);
```

你认为这是C程序还是C++程序？

实际上这是C程序的一部分！

11

```
#include <stdio.h>  
typedef struct A {  
    int z;  
    int (*Get)(struct A);    /* 函数指针 */  
    int (*Put)(struct A *, int); /* 函数指针 */  
} A;  
int get(A a)  
{ return a.z;  
}  
int put(A *a, int z)  
{ a->z=z;  
  return a->z;  
}  
void main( )  
{ A a={2, get, put}; int x;  
  x = a.Get(a);  
  a.Put(&a, x+1);  
  printf("%d\n", a.Get(a));  
}
```

12

第1次作业：

见网络学堂第1次作业

2题必做，1题选做

13

1. 编写程序，从文本文件中读入一篇英文，统计这篇文章中所使用的单词的个数与词频，并按词频从高到低依次逐行列出本文所使用的单词与词频（词频相同时单词按字典序列出）。

说明：英文段落中包含各种标点符号，某些单词开头字母可能大写或者全单词大写，文中也可能包含数字等，请尽量考虑各种英文的实际情况，使得你编写的程序具有通用性。

14

2. 编写程序，从一个二进制文件中读入若干字节，将每一个字节的8位逆转后，按字节的输入顺序输出到另一个二进制文件中。这里的逆转是指：当
`unsigned char k=0xAC`(二进制值为：10101100)
逆转后k的值为：0x35(二进制值为：00110101)
并考虑当文件中的数据量很大时（比如，几百MB, 几百GB甚至几百TB），如何优化此程序，使其执行效率最高。写出优化思路并给出优化后的程序。

15

3*。（探究题，选做）编写程序，将一段文字中出现的所有英文阿拉伯数字和中文阿拉伯数字转化成（按照人们习惯的读音）中文。比如：“我去购物中心花2021元买了一块手表”，其中的：“2021”是英文阿拉伯数字，每个数字占一个字节，应该按照读音转化为：“两千零二十一”。也可能写作：“我去购物中心花2 0 2 1元买了一块手表”，其中的“2 0 2 1”是中文格式的阿拉伯数字，每个数字占两个字节，也应该按照读音转化为：“两千零二十一”。又比如：“他花108050元买了一辆新电动车”，其中的：“108050”应该按照人们读音习惯转化为：“十万八千零五十”。转化单位包括：亿、万、千、百、十……
更进一步，可以转化带小数的数字，例如将：“121.14156”转化为：“一百二十一点一四一五六”。

16

如何判断是否是英文阿拉伯数字和中文阿拉伯数字：

```
for(k=0; k<strlen(str); k++)  
    if (str[k]>='0' && str[k]<='9')  
        unsigned char zero[]=" 0 ", nine[]=" 9 ";  
for(k=0; k<strlen(str); k++)  
    if ((str[k]>=zero[0] && str[k]<=nine[0]) &&  
        (str[k+1]>=zero[1] && str[k+1]<=nine[1]))
```

注意：① 在VS上调试程序时，用GBK编码方式（两个字节），不要用Unicode方式（多字节）。

② 不要出现跨字的问题，因为每个汉字占两个字节，顺序向后遍历字符串时，容易出现把第一个汉字的第二个字节与第二个汉字的第一个字节当作一个汉字来判断，出现跨字错误。

17

C++程序设计

18

C++及创始人简介

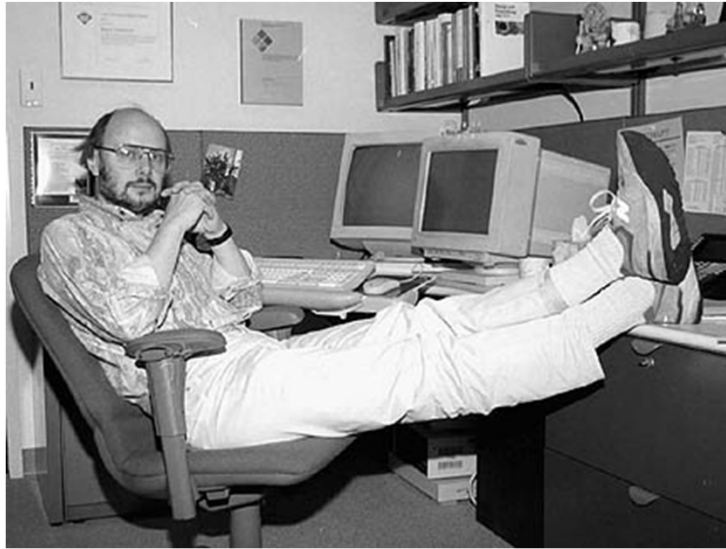
- 1979年，刚从英国剑桥大学（CU）获得博士学位的29岁丹麦人Bjarne Stroustrup (本贾尼·斯特劳斯特卢普)。进入美国AT&T公司Bell(新泽西)实验室，开始在C语言的基础上研制C++，1983年研制出了C++的雏形。
- 最初在1980年被称为：带类的C (C with Class)，1983年正式命名为C++。从1989年开始C++的标准化工作，1994年推出了ANSI C++标准。
- 1998年11月ISO批准为ISO C++ 标准: C++98 (ISO/IEC 14882) 这是C++第一个国际标准
- 2011年9月ISO批准为ISO C++标准: C++11 (ISO/IEC 14882:2011)
- 2020年ISO批准为ISO C++标准: C++20 (ISO/IEC 14882:2020)
- 即将推出下一个ISO C++标准: C++23

19

C++及创始人简介(2)

- Bjarne Stroustrup 在AT&T 的Bell实验室解散后，一度担任美国Texas A&M大学计算机系的特聘教授。
- 近些年他担任过摩根士坦利（Morgan Stanley）技术部的Technical Fellow和Managing Director，兼任哥伦比亚大学计算机系的客座教授。现在是哥伦比亚大学计算机系的教授。
- 他用自己的名字注册了网络域名，个人主页：
<http://www.stroustrup.com/>
- 他的个人感言：
I have stuck with C++ ISO standards effort for 30+ years (so far).

20



Bjarne Stroustrup在AT&T时的英姿

21

Bjarne Stroustrup's Homepage x +

← → ↻ stroustrup.com 🔍 ⚙️ ☆ 📄 👤

🗺️ 地图 📧 Gmail 🗣️ 翻译 📊 DB-Engines Rank...

[home](#) | [C++](#) | [FAQ](#) | [technical FAQ](#) | [publications](#) | [WG21 papers](#) | [TC++PL](#) | [Tour++](#) | [Programming](#) | [D&E](#) | [bio](#) | [interviews](#) | [videos](#) | [quotes](#) | [applications](#) | [guidelines](#) | [compilers](#)

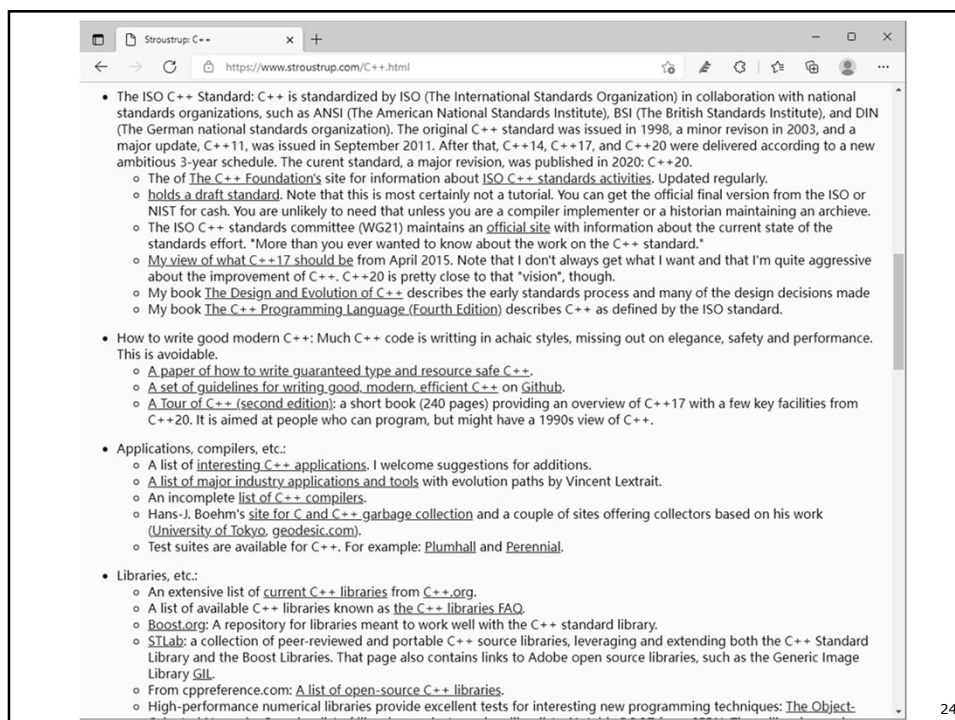
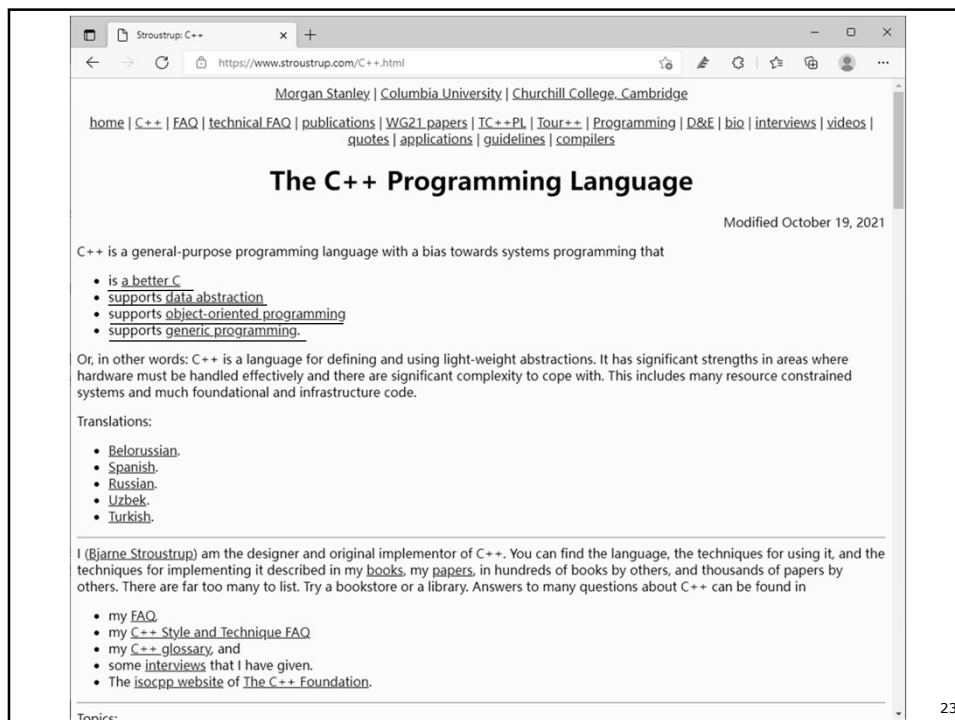
Welcome to Bjarne Stroustrup's homepage!



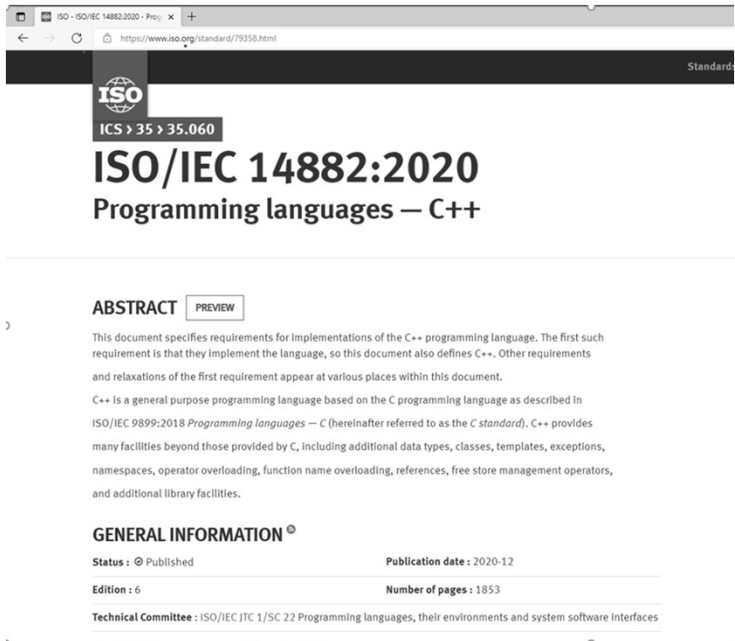
I designed and implemented [the C++ programming language](#). To make C++ a stable and up-to-date base for real-world software development, I have stuck with its ISO standards effort for 30+ years (so far). I'm a professor of Computer Science in Columbia University in New York City.

- [A Tour of C++ \(3rd edition\)](#): a brief (254 pages + index, historical information, etc.) tour of the C++ Programming language and its standard library for experienced programmers.
- [The C++ Programming Language \(4th edition\)](#): an exhaustive description of the C++ Programming language, its standard library, and fundamental techniques for experienced programmers.
- [Programming: Principles and Practice using C++ \(2nd edition\)](#): a programming text book aimed at beginners who want eventually to become professionals.
- [The Design and Evolution of C++](#): a book presenting the rationale and design criteria for C++ and its evolution up until 1994.
- [Research and popular papers](#)
- [Technical reports and proposals for the ISO C++ Standard](#)
- [Videos](#)
- [Interviews](#)
- [Biographical material](#)

22



https://www.iso.org/standard/79358.html



ISO/IEC 14882:2020
Programming languages — C++

ABSTRACT PREVIEW

This document specifies requirements for implementations of the C++ programming language. The first such requirement is that they implement the language, so this document also defines C++. Other requirements and relaxations of the first requirement appear at various places within this document.

C++ is a general purpose programming language based on the C programming language as described in ISO/IEC 9899:2018 *Programming languages — C* (hereinafter referred to as the *C standard*). C++ provides many facilities beyond those provided by C, including additional data types, classes, templates, exceptions, namespaces, operator overloading, function name overloading, references, free store management operators, and additional library facilities.

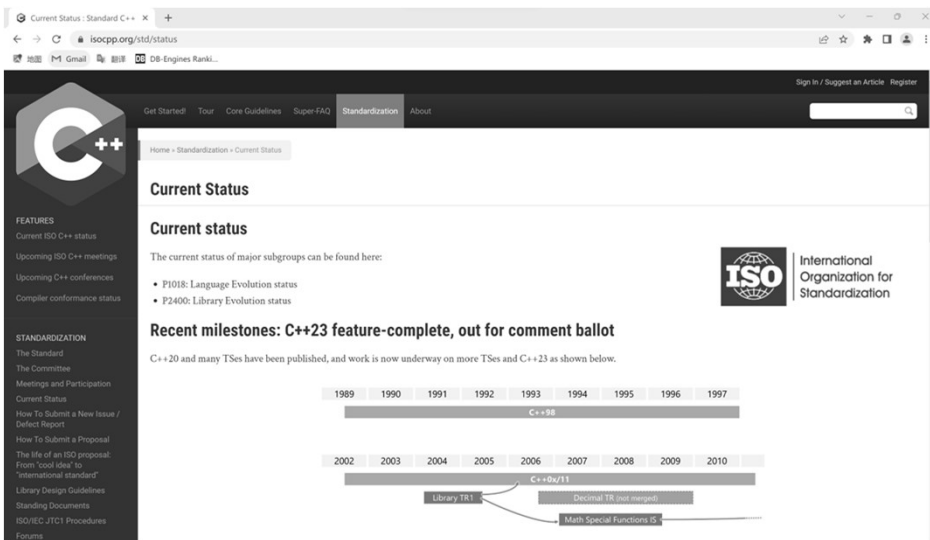
GENERAL INFORMATION

Status: Published **Publication date:** 2020-12

Edition: 6 **Number of pages:** 1853

Technical Committee: ISO/IEC JTC 1/SC 22 Programming languages, their environments and system software interfaces

25



Current Status

Current status

The current status of major subgroups can be found here:

- P1018: Language Evolution status
- P2400: Library Evolution status

Recent milestones: C++23 feature-complete, out for comment ballot

C++20 and many TSes have been published, and work is now underway on more TSes and C++23 as shown below.

Timeline:

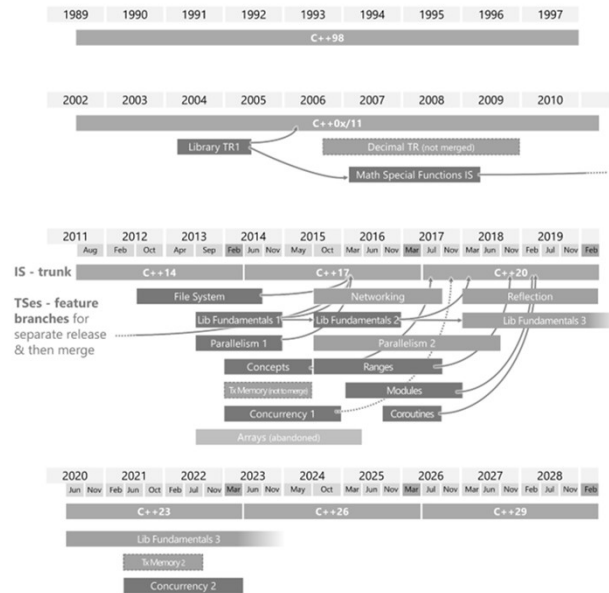
- 1989: C++98
- 2002: C++98/11
- 2003: Library TR1
- 2005: C++98/11
- 2006: C++98/11
- 2007: C++98/11
- 2008: C++98/11
- 2009: C++98/11
- 2010: C++98/11
- 2011: C++98/11
- 2012: C++98/11
- 2013: C++98/11
- 2014: C++98/11
- 2015: C++98/11
- 2016: C++98/11
- 2017: C++98/11
- 2018: C++98/11
- 2019: C++98/11
- 2020: C++98/11
- 2021: C++98/11
- 2022: C++98/11
- 2023: C++98/11

26

网络学堂中，为大家提供了ISOIEC_14882_C++11的pdf版，供大家参考。

Recent milestones: C++23 feature-complete, out for comment ballot

C++20 and many TSes have been published, and work is now underway on more TSes and C++23 as shown below.



27

C++及创始人简介(3)

C++被称为面向对象的程序设计语言

OOPL (Object-Oriented Programming Language)

C是C++的子集，C++是C的超集，C++保持了C的原始思想，程序员写程序时可以用也可以不用C++所增加的功能。

C++在C的关键字的基础上增加了：catch, class, const, delete, friend, inline, new, operator, private, protected, public, template, this, throw, try, typeid, virtual, volatile 等关键字。

C++在C语言中增加的最重要的机制有三个：

- 类 (class)
- 函数名重载 (function name overloading)
- 操作符重载 (operator overloading)

28

C++及创始人简介(4)

主要目的:

1. 用于大型软件(指源程序超过5万行)的工业化软件开发。
2. 解决C语言程序访问权限不受任何限制而导致的副作用严重的问题:
 - a. 全局变量全局可见可任意访问修改
 - b. 不检查数组越界引起的致命错误
3. 生成严格控制存取权限的“黑盒子功能单元”
---对象 (object)

29

C++创始人对C++的点评

C++是一种通用编程语言，偏向于系统编程。

1. C++是更好的(改进的) C语言;
2. 支持数据抽象(data abstraction);
3. 支持面向对象程序设计(object-oriented programming);
4. 支持泛型程序设计(generic programming)。

换句话说，C++是一种定义和使用轻量级抽象的语言。它在必须有效处理硬件和处理复杂问题的领域具有显著优势。这包括许多资源受限的系统和许多基本的和基础的结构代码。

30

ISO对C++20的简介

- C++ (ISO/IEC 14882:2020)是一种基于C (ISO/IEC 9899:2018)程序设计语言的通用编程语言。
- C++提供了许多C所没有的功能，包括：
更多的数据类型(data type)、类(class)、模板(template)、异常(exception)、命名空间(namespace)、运算符重载(operator overloading)、函数名重载(function name overloading)、引用(references)、自由存储管理运算符(free store management operator)，以及额外的库功能(additional library facilities)。

31

1. 关于C++的几个基本问题

- 1.1 几个基本概念
- 1.2 基本数据类型
- 1.3 注释
- 1.4 输入/输出
- 1.5 内联函数(inline)
- 1.6 动态内存分配
- 1.7 指针/地址的传递 --- 引用(&)
- 1.8 共用数据的保护(const)
 - 1.8.1 常对象
 - 1.8.2 常数据成员
 - 1.8.3 指向对象的常指针
 - 1.8.4 指向常对象的指针
 - 1.8.5 指向常对象的常指针
- 1.9 作用域与可见性
- 1.10 命名空间(namespace)
 - 1.10.1 命名空间与作用域
- 1.11 缺省参数

32

1.1 几个基本概念

① 数据抽象(Data abstraction)

- 将数据类型与操作分开，被称为数据抽象。

例如：栈操作，操作对象的类型可以是int, long, float, double等等，但不考虑类型，就可以使之成为一个数据抽象---栈操作，可以由此写出一个栈操作模板，适用于任何数据类型的栈操作。

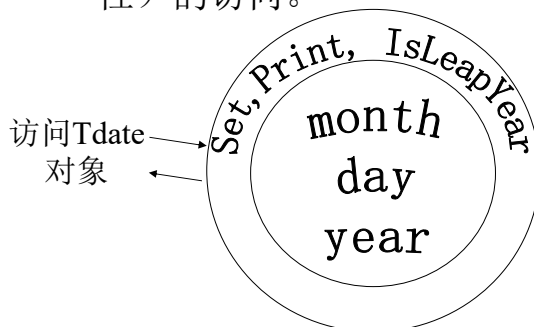
```
T abs(T i)
{   return i<0 ? -i : i; }
```

abs是一个函数模板(function template)，可对任意类型的数据求绝对值，只要此数据可以和0比较大小。

33

② 对象(Object)

- 将一个(组)变量（属性），以及作用于这些变量的一个(组)函数（行为）封装为一个单元，这个单元就被称为对象。类是同类对象的抽象(共同特性的描述)，对象是类的实例化。
- 对象具有封闭性，可以自己控制外界对它的访问权限，从而严格控制外界对其中被封装变量（属性）的访问。



```
class Tdate
{ public:
    void Set(int d, int m, int y);
    int IsLeapYear( );
    void Print( );
private:
    int day, month, year;
};
```

34

③ 面向对象程序设计(Object-Oriented Programming)

- 面向对象程序设计(OOP)的实质是选用一种面向对象程序设计语言(OOPL)，采用对象、类及其相关概念所进行的程序设计。面向对象的方法主要是将事物对象化，每个对象包括属性与行为。

对应：面向过程的程序设计（C语言）

面向过程其实是最为实际的一种思考方式，其实面向对象的方法也是含有面向过程的思想。可以说面向过程是一种基础的方法。它考虑的是实际的实现。一般的面向过程是从上往下逐步求精。所以面向过程最重要的是模块化的思想方法。当程序规模不是很大时，面向过程的方法还会体现出一种优势，因为程序的流程很清楚，按模块与函数的方法可以很好的组织编写程序。

35

④ 泛型程序设计(Generic programming)

泛型程序设计是指将程序写得尽可能通用，将算法从特定的数据结构和类型中抽象出来，使之成为通用的算法(模板)。便于程序复用，便于开展大规模工业化软件开发。模板是泛型程序设计的基础。

泛型是程序设计语言的一种特性。允许程序员在强类型程序设计语言中编写代码时定义一些可变部分，那些部分在使用前必须先指明。各种程序设计语言和其编译器、运行环境对泛型的支持均不一样。将类型参数化以达到代码复用提高软件开发工作效率的一种数据类型。

36

C++ 的泛型

C++ 无法对泛型的类型参数进行约束。在编译时，每个被使用的封闭泛型类型（即所有泛型参数的实际类型都已被指明的泛型）都会有独立的编码产生。

比如：由函数模板生成相应多个模板函数，由类模板生成相应多个模板类，进而定义相应模板类的对象。而类模板中可以包含多个类成员函数模板，在生成相应模板类的同时，也会生成相应的类成员模板函数。也就是说，在编译时中先生成C++源代码，再将C++源代码编译成二级制机器指令，编译器会在生成C++源代码的过程中确保类型安全性。

C++标准模板库(STL)就是C++ 泛型程序设计的范例。

37

1.2 基本数据类型

C++全面继承了C的基本数据类型：char, short, int, long, float, double, long long等，并引入了新的类型：

bool（布尔型）取值只有两个：true/false

给bool变量赋值别的数，都将强制转换成true(不为0时)或 false(为0时)。

bool型占1个字节，等同于char类型。

例如，有程序：

38

```
#include <stdio.h>
void main( )
{ bool f1, f2; int i;
  f1=true; f2=false;
  printf("%d %d\n", f1, f2);
  i = (f1+1)* 10;
  f2=i;
  printf("%d %d %d\n", sizeof(f1), i, f2);
}
```

编译时给出警告：

test.cpp(7) : warning C4800: “int”: 将值强制为布尔值 “true”或 “false”(性能警告)

运行结果为：

1 0

1 20 1

请按任意键继续...

分析：bool型变量占一个字节，赋值时会进行强制类型转换。

39

1.3 注释

C语言用 /* 和 */ 进行注释，从 /* 开始一直到 */ 之间为注释信息。

C++程序中增加了**行注释**：//，从 // 开始到本行尾（回车）为注释信息。

例如：

```
void main( )
{ int i;          // i是循环变量
  double sum;    // sum用来求累加和
  .....
}
```

40

1.4 输入/输出

C++利用cin和cout这两个流类对象进行输入/输出，完全是自由格式。例如：

```
cin >> i >> f; // 等同于 scanf("%d%f", &i, &f);
cout << i << f << endl;
// 等同于 printf("%d%f\n", i, f);
// 其中 endl 等同于 '\n'
```

源程序开头的文件引用改为：

```
#include <iostream.h>
```

而不是C源程序所文件引用的：

```
#include <stdio.h>
```

41

源程序开头的文件引用：

```
#include <iostream.h>
```

也可以写成：

```
#include <iostream>
```

```
using namespace std;
```

这是目前ISO C++建议的写法，意思是：使用标准命名空间，这样#include所引用的头文件将是不带.h的。

原来C语言的stdio.h, string.h, math.h等头文件引用仍可使用，但需要去掉文件名后缀.h，前面加上c：

```
#include <cstring>
```

```
#include <cmath>
```

对于一般的输入输出操作上面两种写法没有本质区别，但文件操作将会有较大差别。

42

ISO C++ 写法:

```
#include <iostream>
using namespace std;
void main( )
{ int i=33, j=35, k=12345;
  float f=3.14159f;
  cout << i << j << endl;
  cout << k;
  cout << i << j << endl;
  cout << j << f << endl;
}
```

输出结果:

3335

123453335

353.14159

43

也可以写为: (这个写法最标准)

```
#include <iostream>
using namespace std;
int main( )
{ int i=33, j=35, k=12345;
  float f=3.14159f;
  cout << i << j << endl;
  cout << k;
  cout << i << j << endl;
  cout << j << f << endl;
  return 0; // 主函数的返回值, 这个返回值任意
}
```

输出结果:

3335

123453335

353.14159

44

1.4 输入/输出(续1)

常用的I/O流类库操纵符(函数):

`setw(int)` 设置随后(一个)输出内容的场宽, 不够自动突破

`setprecision(int)` 设置随后输出的(一个)浮点数的位数(不包括小数点)

`hex` 随后所有的整型数值采用十六进制输出

`oct` 随后所有的整数值采用八进制输出

`dec` 随后所有的整数值采用十进制输出

`endl` 回车符

要想使用`setw`和`setprecision`, 还必须引用 `iomanip.h`

程序开始处加: `#include <iomanip.h>`

或 `#include <iomanip>`

45

```
#include <iostream>
#include <iomanip>
using namespace std;
void main(void)
{ int i=33, j=35, k=12345;
  float f=3.14159f;
  cout << setw(10) << i << setw(8) << j << endl;
  cout << setw(3) << k << endl; // 场宽不足,自动突破
  cout << hex << i << setw(3) << j << endl; // 用十六进制输出整数
  cout << i << setw(3) << j << endl;
  cout << oct << i << setw(j/8) << j << endl; // 用八进制输出整数
  cout << setw(12) << setprecision(5) << f << endl;
  cout << setw(4) << setprecision(6) << f << endl; //场宽不足,自动突破
  cout << setprecision(4) << f << endl;
      // 未定场宽,按照实际宽度输出
  cout << setprecision(1) << f << endl;
  cout << setprecision(2) << 314.159 << endl;
}
```

46

运行结果:

```
33 35
12345
21 23
21 23
41 43
3.1416
3.14159
3.142
3
3.1e+002
```

47

1.4 输入/输出 字符串

```
char s1[20], s2[20], s3[20];
cin >> s1 >> s2 >> s3;
// 等同于 scanf("%s%s%s", s1, s2, s3);
cout << s1 << s2 << s3 << endl;
// 若从键盘上输入: How are you?
// 输出结果是: Howareyou?
整行输入字符串:
cin.getline(s1, N, 结束符); // '\n' 为默认结束符
cin.get(s2, N, 结束符); // 一次连续读入多个(包括空格)
// 字符, 直到读满N-1个, 或遇到结束符
// getline读取但不存结束符, get既不读取也不存结束符
```

48


```

#include <iostream>
using namespace std;
void main()
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.get(s2, 20, '\n');
  cin.get(s3, 20, '\n');
  cout << s1 << s2 << s3;
}

```

运行结果：

abcd

efgh

abcdefgh请按任意键继续...

可以看到get读s2时未读掉回车符，导致读s3又遇到回车符，未读入内容，s3为空字符串。

49

```

#include <iostream>
using namespace std;
void main()
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.getline(s2, 20, '\n');
  cin.getline(s3, 20, '\n');
  cout << s1<<"==" << s2 <<"=="<< s3<<"==";
}

```

运行结果：

123456789012345678901234567890

1234567890123456789=====请按任意键继续...

可以看到getline读入了19个字符给s1，但并没有把这一行剩余的字符读入给s2，s3也未读入字符，为什么？？？

因为getline读入19个字符给s1时没有遇到结束符，因而产生错误，cin处于错误状态，需要清除错误状态后才能继续读入。

50

```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.clear( ); //清除cin输入对象的错误状态
  cin.getline(s2, 20, '\n');
  cin.getline(s3, 20, '\n');
  cout << s1<<"==" << s2 <<"=="<< s3<<"==";
}
```

运行结果：

123456789012345678901234567890

123456789012345

1234567890123456789==01234567890==123456789012345==请
按任意键继续...

可以看到getline读入了19个字符给s1，又把同一行剩余的字符直到'\n'读入给s2，并继续等待输入，又读入字符串给s3。

51

```
#include <iostream>
using namespace std;
void main( )
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20, '\n'); // 最多读入19个字符
  cin.clear( ); //清除cin输入对象的错误状态
  cin.get(s2, 20, '\n'); // getline 改为get
  cin.getline(s3, 20, '\n');
  cout << s1<<"==" << s2 <<"=="<< s3<<"==";
}
```

运行结果：

123456789012345678901234567890

1234567890123456789==01234567890====请按任意键继续...

可以看到把第二个getline改为get后，getline读入了19个字符给s1，get把同一行剩余的字符直到'\n'读入给s2，但并没有读掉'\n'，getline读入空字符串给s3。

52

1.5 内联函数(inline)

为提高执行效率，C++增加了内联函数。

与宏定义`define`类似，编译器将出现函数名的位置用函数体代码进行替换，即所谓在线化，这样省去了函数调用的参数入栈、退栈、跳转等所花费的时间和空间。

内联函数比`define`方便，功能也更强，不只是简单的**字符串的替换**，全面考虑了**类型匹配**问题。

例如：

53

```
#include <iostream>
using namespace std;
inline double Area(double R) // 根据半径R求圆面积
{ return 3.14159*R*R; }
void main( )
{ double r(3.0); // 赋初值，等同于 double r=3.0;
  double area;
  area = Area(r); // 执行效率等同于 area=3.14159*r*r;
  cout << area << '\n';
}
运行结果： 28.2743
```

54

1.5 内联函数(inline) (续)

内联函数的书写有一些限制 (但没有统一的明确规定, 由各家的编译器的聪明程度决定):

- 内联函数中可以定义变量, 可以使用循环语句、分支语句等;
- 但内联函数不能直接递归或间接递归;
- 其它使编译器不能把函数调用在线化的情形。

凡是编译器没法把函数调用在线化的内联函数, 编译器都会自动忽略**inline**, 把它当做一般函数来调用, **inline**也就自动失效了。

55

```
#include <iostream>
using namespace std;
inline double Area(double R)
{ if (R>0)
    return Area(R-1) + 3.14159*R*R;
  return 3.14159*R*R;
}
void main( )
{ double r(3.0); double area;
  area = Area(r);
  cout << area << '\n';
}
```

如果把前面例子的Area函数改为自递归函数, 照样可以编译运行, 运行结果: 43.9823

但实际上Area并没有被当做内联函数。

56

1.6 动态内存分配

C++用new和delete操作符 分配和释放动态内存，
等同于C的malloc和free函数，但new和delete功能更强大。

例如： `int *p=new int(10);`
//分配1个 int 动态内存单元,并赋初值为10

等同于C的：

```
int *p=(int *)malloc(sizeof(int));  
*p = 10;
```

C++用：

```
delete p; // 释放p所指的1个int单元
```

等同于C的：

```
free(p);
```

57

1.6 动态内存分配(续1)

```
int *pt=new int[10]; // 分配10个int单元，  
// 即pt是一个动态数组，pt[0]~pt[9]
```

等同于C的：

```
int *pt = (int *)malloc(sizeof(int)*10);
```

C++用：

```
delete []pt;
```

释放pt所指的10个连续int单元， 等同于C的：

```
free(pt);
```

提醒： 注意 `pt=new int(10);` 与 `pt=new int[10];` 的差别

58

1.6 动态内存分配(续2)

```
int (*pt)[10]=new int[8][10]; //分配80个int单元  
    // pt是一个二维动态数组，pt[0][0]~pt[7][9]  
等同于C的：
```

```
int (*pt)[10];
```

```
pt = (int (*)[10])malloc(sizeof(int)*8*10);
```

C++用：

```
delete []pt;
```

释放行指针pt所指的80个连续int动态内存单元，
等同于C的：

```
free(pt);
```

59

1.6 动态内存分配(续3)

```
int **q = new int *[10]; //分配10个int指针单元  
// q是一个一维动态指针数组，q[0]~q[9]  
等同于C的：
```

```
int **q;
```

```
q = (int **)malloc(sizeof(int *)*10);
```

C++用：

```
delete []q;
```

释放q所指的10个连续int指针单元，等同于C的：

```
free(q);
```

60

例如：

```
#include <iostream>
using namespace std;
void main( )
{ int **q = new int *[10];
  int a[10]={1,2,3,4,5,6,7,8,9,10}, k;
  for (k=0; k<10; k++)
    q[k] = &a[k];
  for (k=9; k>=0; k--)
    cout << *q[k] << ',';
  delete [ ]q;
}
```

运行结果：

10,9,8,7,6,5,4,3,2,1,请按任意键继续...

61

如何用new实现C函数realloc的功能：

```
#include <iostream>
using namespace std;
void main() // 动态内存长度随着读入int个数的增加而增加
{ int *a,*b, n=1,k,i;
  a = new int[n]; // 先申请一块长度为1个int的动态内存
  cin >> k;
  while (k>=0) // 读入整数序列，直到输入一个负数结束
  { a[n-1]=k;
    b = new int[n+1]; // 申请一块更大的动态内存
    for (i=0; i<n; i++) // 复制原有数组中的元素值到新数组中
      b[i]=a[i];
    delete []a; // 释放原来的动态内存
    n++;
    a = b; // 以新申请的动态内存替代原来的动态内存
    cin >> k;
  }
  for (i=0; i<n-1; i++)
    cout << a[i] << ",";
  delete []a; // 释放a所指的长度为n个int的动态内存
}
```

62

强调一下：

- ① `new`和`delete`是C++的操作符，不是函数，这与`malloc`和`free`完全不同。
- ② `new`和`delete`操作符的功能更强大，不仅仅是按指定大小申请动态内存。

`new`可以触发相应对象（对象数组）的构造函数的一次或多次执行，构造一个或多个（数组）动态对象并初始化，并可能会进一步触发更多`new`操作。

`delete`可以触发相应对象（对象数组）的析构函数的一次或多次执行，析构掉一个或多个（数组）动态对象，并可能会进一步触发更多`delete`操作。

63

强调一下：

- ③ 最新版本的编译器中（VS2019以上），`new`操作符申请动态内存后可以同时赋初值：

```
int *pt=new int[10]{1,2,3,4,5,6,7,8,9,10};  
    // 建立pt动态数组并赋初值：1,2,3,4,5,6,7,8,9,10  
int (*pd)[5]=new int[4][5] {{01,02,03,04,05},  
                             {11,12,13,14,15},  
                             {21,22,23,24,25},  
                             {31,32,33,34,35}};  
    // 建立4×5的pd二维动态数组并赋初值
```

64

1.7 指针/地址的传递---引用(&)

- ◆ C的参数传递只是**传值**,无法将变量地址传递到函数中。除非通过显式的指针方式传递。
- ◆ C++提供了**引用(reference)**机制来**传递变量地址**。

方法是:

- ◆ 在函数头部要传递地址的形参名前加**&**，而函数调用方式不变。
- ◆ 这样在被调用函数中对声明**传递变量地址**的变量的赋值操作，实际上是直接对调用函数中相应变量的操作。

65

例如：交换两个变量的值(C版本)

```
#include <iostream>
using namespace std;
void swap(int a, int b) // 将a, b变量的值交换
{   int t=a;
    a=b; b=t;
}
void main( )
{   int i=3, j=5;
    swap(i, j);    //交换i, j的值
    cout << i << "    " << j << endl;
}
输出结果将是： 3 5    a和b的值并未交换
```

66

在C语言方式下除非改成传递地址的指针方式：

```
#include <iostream>
using namespace std;
void swap(int *a, int *b) // 将a,b所指变量的值交换
{   int t=*a;
    *a = *b; *b=t;
}
void main( )
{   int i=3, j=5;
    swap(&i, &j); //交换i, j的值
    cout << i << "    " << j << endl ;
}
输出结果将是： 5 3    // a和b的值已经交换
```

67

C++引用声明传地址方式：

```
#include <iostream>
using namespace std;
void swap(int &a, int &b) // 引用声明传递a,b变量的地址
{   int t=a;
    a=b; b=t;
}
void main( )
{   int i=3, j=5;
    swap(i, j);    //交换i, j的值
    cout << i << "    " << j << endl ;
}
输出结果将是： 5 3    // a和b的值已经交换
```

68

又例如：交换两个指针变量的值

```
#include <iostream>
using namespace std;
void swap(int *a, int *b) // 将a, b指针变量的值交换
{ int *t;
  t=a; a=b; b=t;
}
void main( )
{ int i=3, j=5, *p=&i, *q=&j; // p指向i, q指向j
  swap(p, q); //交换p, q指针的值, 让p指向j, q指向i
  cout << *p << " " << *q<<endl ;
}
输出结果将是： 3 5 // p和q的值并未交换
```

69

除非传递指针变量的地址：

```
#include <iostream>
using namespace std;
void swap(int **a, int **b) // 将a,b所指变量的值交换
{ int *t;
  t=*a; *a=*b; *b=t;
}
void main( )
{ int i=3, j=5, *p=&i, *q=&j; // p指向i, q指向j
  swap(&p, &q); //交换p,q指针的值,让p指向j,q指向i
  cout << *p << " " << *q<<endl ;
}
输出结果将是： 5 3 // p和q的值已交换
```

70

例如, 此max函数返回的是引用:

```
#include <iostream>
using namespace std;
const int &max(int &a, const int &b)
{   return a>b? a:b;
    // 返回的是a或b也就是i或j的地址
}
void main( )
{   int i=3, j=5, k;
    k=max(i, j); //取i, j的值
    cout << k << endl ;
}
```

73

1.7 指针/地址的传递---引用(&) (续1)

引用(reference)是一种特殊类型的变量, 可以认为是给一个变量起别名。

```
int i, j;
```

```
int &r=i; // 变量引用, r和i是同一个内存单元
```

// r不能再改变引用, 可以认为r是i的别名

```
int *p=&i; // 注意指针与引用的区别
```

```
i=10;
```

```
j=r;           // 等价于 j=i
```

```
*p = 20;       // 等价于 i=20
```

74

1.7 指针/地址的传递---引用(&) (续2)

```
#include <iostream>
using namespace std;
void main( )
{ int i=10, j=20, k=30, *p=&i;
  int *&q=p; //指针引用, q和p是同一个指针单元
  // q不能再改变引用, 可以认为q是p的别名
  cout << *p << ' ' << *q << endl;
  p = &j;
  cout << *p << ' ' << *q << endl;
  q = &k;
  cout << *p << ' ' << *q << endl;
}
```

运行结果为:

```
10 10
20 20
30 30
```

75

1.8 共用数据的保护(const)

- ◆ **const**是常类型修饰符, 在所定义的变量类型前面或后面出现, 使之成为不可修改的量: **常量**。
例如, `int const a=1;` 或 `const int a=1;`
使变量a成为不可修改的量, 等同于C的常量, 但 **const**比C语言的常量功能更强, 因为**const**是带类型的。
- ◆ **const**经常用于函数参数说明, 防止在函数中无意破坏参数的值或所指内容。
例如, `strcpy(char *str2, const char *str1);`
是将字符串str1复制到str2。将参数str1定义为**const**, 就是防止在**strcpy**函数中破坏str1所指的字符串。
- ◆ 尤其是在C++引入传地址的引用机制后, **const**用来保证在函数中不会修改相应参数变量的值或所指内容。

76

例如：

```
#include <iostream>
using namespace std;
void swap(int &a, const int &b) // b是常引用
{ int t=a;
  a=b;
  b=t;          // 第6行
}
void main( )
{ int i=3, j=5;
  swap(i, j);    //交换i, j的值
  cout << i << " " << j << endl;
}
```

编译结果：

swap_const.cpp(6) : error C3892: “b”: 不能给常量赋值

77

又例如：

```
#include <iostream>
using namespace std;
void swap(int &a, const int &b) // b是常引用
{ int t=a; a=b;
  b=t;    // 第5行
}
void main( )
{ int const i=3, j=5; // i和j是整型常量
  i=4;    // 第9行
  j=6;    // 第10行
  swap(i, j); //交换i, j的值
  cout << i << " " << j << endl;
}
```

编译结果：

test.cpp(5) : error C3892: “b”: 不能给常量赋值
test.cpp(9) : error C3892: “i”: 不能给常量赋值
test.cpp(10) : error C3892: “j”: 不能给常量赋值

78

1.8.1 常对象

例如:

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int  sno;
};
const STU a={"Zhang San", 'm', 2021000001}; // 常对象
// 也可以写为: STU const a={"Zhang San", 'm', 2021000001};
void main() // C++结构体类型定义变量不必再写struct
{ STU b={"Li Shuang", 'f', 2021000002};
  b=a;
  cout<<b.name<<" "<<b.gender<<" "<<b.sno<<" "<<endl;
}
编译没有错误, 运行结果:
Zhang San m 2021000001
```

79

如果将语句: `b=a;`
改为: `a=b;`

编译结果:

test.cpp(11) : error C2678: 二进制 “=”: 没有找到接受 “const STU” 类型的左操作数的运算符(或没有可接受的转换)

1> test.cpp(7): 可能是 “STU &STU::operator =(const STU &)”

1> 试图匹配参数列表 “(const STU, STU)”时

因为a是一个常量对象, 简称为: 常对象

除了定义时一次性赋初值, 不能再改变它的值, 从而共用时起到保护作用。

80

1.8.2 常数据成员

例如:

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    static int const sno=1; // 静态常数据成员
};
STU const a={"Zhang San", 'm'}; // 常对象
void main( )
{ STU b={"Li Shuang", 'f'}; // 不能再给sno赋值
  b=a;
  cout<<b.name<<" "<<b.gender <<" "
    <<b.sno <<" "<<endl;
}
```

编译没有错误, 运行结果:

Zhang San m 1

81

但是结构体中成员sno前面的static不能去掉, 否则在VS2008编译器上会出现编译错误:

```
test.cpp(6) : error C2864: "STU::sno": 只有静态常量整型数据成员才可以在类中初始化
test.cpp(8) : error C3852: "STU::sno"具有类型 "const int": 聚合初始化未能初始化该成员
1> 常量成员不能被默认初始化, 除非它们的类型具有用户定义的默认构造函数
test.cpp(8) : error C2512: "STU::STU": 没有合适的默认构造函数可用
test.cpp(8) : error C2078: 初始值设定项太多
test.cpp(10) : error C3852: "STU::sno"具有类型 "const int": 聚合初始化未能初始化该成员
1> 常量成员不能被默认初始化, 除非它们的类型具有用户定义的默认构造函数
test.cpp(10) : error C2512: "STU::STU": 没有合适的默认构造函数可用
test.cpp(10) : error C2078: 初始值设定项太多
test.cpp(11) : error C2582: "operator ="函数在 "STU"中不可用
```

编译器提示: 只有静态常量整型数据成员才可以在类中初始化。

82

1.8.3 指向对象的常指针

例如：

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
STU a={"Zhang San", 'm', 2021000001};
STU * const p = &a; // 常指针，此常指针p只能初始化时赋值
void main()
{ STU b={"Li Shuang", 'f', 2021000002};
  p=&b; // 第12行
  cout<<p->name<<" "<<p->gender<<" "<<p->sno<<" "<<endl;
}
编译结果：test.cpp(12) : error C3892: “p”: 不能给常量赋值
```

83

1.8.4 指向常对象的指针

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
const STU a={"Zhang San", 'm', 2021000001};
STU * const p = &a; // 第9行 常指针不能指向常对象
void main()
{ STU b={"Li Shuang", 'f', 2021000002}, *q;
  q=&a; // 第12行 普通指针也不能指向常对象
  cout<<p->name<<" "<<p->gender<<" "<<p->sno <<" "<<endl;
}
编译结果：
test.cpp(9) : error C2440: "初始化": 无法从"const STU *"转换为"STU *const"
           转换丢失限定符
test.cpp(12) : error C2440: “=”: 无法从 “const STU *”转换为 “STU *”
           转换丢失限定符
```

84

改为:

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
const STU a={"Zhang San", 'm', 2021000001};
const STU * p = &a;    // 第9行 p是指向常对象的指针
void main()
{ STU b={"Li Shuang", 'f', 2021000002};
  p=&b; // 第12行 p不是常指针, 可以被修改
  cout<<p->name<<" "<<p->gender<<" " <<p->sno <<" "<<endl;
}
```

编译没有任何错误, 运行结果:

Li Shuang f 2021000002

结论: 指向常对象的指针不是常指针, 可以被修改, 也可以指向普通对象。

85

1.8.5 指向常对象的常指针

```
#include <iostream>
using namespace std;
struct STU{
    char name[10];
    char gender;
    int sno;
};
const STU a={"Zhang San", 'm', 2021000001}; // 常对象
const STU * const p = &a; // 第9行 p是指向常对象的常指针
void main()
{ STU b={"Li Shuang", 'f', 2021000002};
  p=&b; // 第12行 p是常指针, 不能被修改
  cout<<p->name<<" "<<p->gender<<" " <<p->sno <<" "<<endl;
}
```

编译结果: test.cpp(12): error C3892: "p": 不能给常量赋值

此外还有: 常成员函数 void get() **const** { } 等讲到类时再解释。

86

1.9 作用域与可见性

- 在C语言中可以出现局部变量和全局变量同名，但函数中的局部变量将掩蔽全局变量（mask），直到局部变量的作用域失效，才能访问同名的全局变量。
- 而实际应用中可能需要同时访问同名的局部变量和全局变量。
- C++为了克服这个缺点，引入了作用域操作符（scope operator）‘::’，使我们可以同时访问同名的局部变量(max)和全局变量(::max)。

例如：下例中max是局部变量，::max是全局变量

87

```
#include <iostream>
using namespace std;
const int max=10000;
void main( ) // 从键盘上读入10个数，求最大值max输出,但最大不超过10000
{ int max, i, j;
  cin >> max;
  for (j=1; j<10; j++)
  {   cin >> i;
      if (i>max) max = i;
      if (max > ::max) max = ::max;
  }
  cout << max<<endl;
}
```

若max大于外部变量max的值，则取外部常量max的值，这种做法通常用来防止数组越界或溢出。

88

全局作用域

文件作用域（静态变量与静态函数）

函数作用域

复合语句作用域

89

1.10 命名空间(namespace)

- 命名空间将不同的标识符（类、对象或函数）定义在一个命名作用域内，主要目的是为了解决大型软件开发中的命名冲突。同时也限制这些标识符的作用域。
- 例如，若声明了一个叫NS的命名空间：

```
namespace NS {  
    class File;  
    void fun();  
}
```

则引用的方式是：

```
NS::File obj; // 定义File类的对象obj  
NS::fun();    // 调用函数fun
```
- 通常，没有命名空间的标识符都处于无名的命名空间中，也就是全局作用域。前一节的`::max`就是这种情况。
- 命名空间可以嵌套定义，即一个命名空间中可以定义更多（子）命名空间。进一步细分标识符的作用域。

90

1.10 命名空间(namespace)

■ 命名空间的定义方法

命名空间定义的一般格式如下：

```
namespace 空间名
{
    变量(可以带有初始化);
    常量;
    函数(可以是定义或声明);
    结构体;
    类;
    模板;
    命名空间(命名空间中嵌套定义子命名空间)
    .....
}
```

91

全局作用域（无名空间）

标准命名空间（**std**）

命名空间**1**

命名空间**2**

.....

命名空间**n**

92

1.10 命名空间(namespace)

- 命名空间可以起别名： `namespace N2 = NS;`
- 可以用 `using` 来指定默认命名空间。
- 比如，若有以下声明：
`using NS::File;`
则在有效作用域内，使用 `File` 定义对象不用加 `NS::`，默认 `File` 是 `NS` 空间的：
`File obj;`
- `using namespace NS;`
就是指定 `NS` 命名空间，则随后的语句可以直接引用 `NS` 中的符号和函数，不用再加 `NS::` 限定名
`File obj;`
`Fun( );`

93

1.10 命名空间(namespace)

- `using namespace std;` 就是指定了 `std` 标准命名空间，`std` 中的所有标识符都可以直接引用。
- 使用标准命名空间，所有引用的头文件都不再使用扩展名 “.h”，比如：
`# include <iostream> // 会出现 ios_base 与 ios 的区别`
`# include <cstring> // C 的 .h 文件去 .h 后前面加 c`
`using namespace std;`
- 若两个命名空间中有同名的标识符，使用时最好加上对应命名空间的限定名，比如：
`NS::a     std::a`
- 否则可能会导致因为打开不同的命名空间而产生不同的结果（有的结果肯定不对）。

94

■ 命名空间的定义和使用举例：

```
#include <iostream>
using namespace std;
int const Num = 1; //全局作用域
namespace Name1
{
    int Num = 8; // Name1命名空间作用域
    int Add(int Num2) // Num2 函数作用域
    {
        Num2 = ::Num + Num + Num2; //::表示无名空间，全局作用域
        return Num2;
    }
}
namespace Name2
{
    int Num = 9; // Name2 命名空间作用域
    int Add(int Num) // Num 函数作用域
    {
        Num = ::Num + Name2::Num + Num;
        return Num;
    }
}
```

95

```
void main(void)
{
    cout << Name1::Add(4) + ::Num << endl;
    cout << Name2::Add(5) + Num << endl;
    namespace N2 = Name1;
    cout << N2::Add(6) + N2::Num << endl;
    using namespace Name2;
    cout << Add(7) + Name2::Num << endl;
}
```

程序运行结果为:

14

16

23

26

请按任意键继续...

说明：Name1命名空间中Add函数中：

Num2 = ::Num + Num + Num2;

是将全局变量Num与当前命名空间中的Num以及函数形参Num2求和。

而Name2命名空间中Add函数中：

Num = ::Num + Name2::Num + Num;

是将全局变量Num与Name2命名空间中的Num以及函数形参Num求和。

96

如果把主程序最后一行改为:

```
cout << Add(7) + Num << endl;
```

编译结果:

test.cpp(29) : error C2872: “Num”: 不明确的符号

1> 可能是 “test.cpp(3) : const int Num”

1> 或 “test.cpp(15) : int Name2::Num”

原因是:

```
cout << Add(7) + Num << endl;
```

语句前面的一行:

```
using namespace Name2;
```

指定Name2为默认命名空间, Add(7) 前不必再加Name2::限定命名空间。但因为Num是全局变量, 同时也是Name2命名空间中的变量, 编译器将无法确定是哪一个Num, 因此必须写为: Name2::Num 指明是Name2命名空间中的Num变量, 写为 ::Num 也可以避免出现编译错误。

97

1.10.1 命名空间与作用域

- 命名空间的引入, 进一步细化了函数和对象的作用域。
- C++作用域由小到大依次为: 复合语句作用域、函数作用域、文件作用域、类作用域、命名空间作用域、全局作用域。

① 复合语句作用域

在任意复合语句{ } 中定义的局部变量, 其作用域仅限在局部小范围内。

② 函数作用域

在函数体内{ } 中定义的局部变量, 其作用域仅限在本函数内。

98

1.10.1 命名空间与作用域

③ 文件作用域

全局变量或函数前加上`static`，使得其作用域仅限于全局变量或函数的源程序所在的文件中。

④ 类作用域

类中所封装的成员变量或成员函数，其作用域仅限于本类对象中。

⑤ 命名空间作用域

将不同的类、对象（变量）或函数定义在一个命名空间作用域内，其作用域仅限于本命名空间。

⑥ 全局作用域

将类、对象（变量）或函数定义为全局变量，其作用域为全程序全局可见。

99

全局作用域

命名空间作用域

类作用域

文件作用域

函数作用域

复合语句作用域

100

1.11 缺省参数

在函数声明中可以为一个或多个参数指定缺省值，这样函数调用时可以从右边开始省略一个或多个参数。例如，若有以下函数说明：

```
void delay(int day=1, int month=3, int year=2022)
{.....}
```

则以下函数调用都是正确的：

```
delay();      // 将使day=1, month=3, year=2022;
delay(29);    // 将使day=29, month=3, year=2022;
delay(29, 4);  // 将使day=29, month=4, year=2022;
delay(2, 5, 2023); // 将使day=2, month=5, year=2023;
为方便函数的使用，C++的库函数参数采用了缺省方式。
比如： cin.getline(s1, 20); // 缺省 '\n' 为结束符
```

101

```
#include <iostream>
using namespace std;
void delay(int day=1, int month=3, int year=2022)
{ cout << month << '/' << day << '/' << year << endl;
}
void main( )
{ delay( );          // 将缺省使day=1, month=3, year=2022
  delay(29);         // 将使day=29, month=3, year=2022
  delay(29, 4);       // 将使day=29, month=4, year=2022
  delay(2,5,2023);    // 将使day=2, month=5, year=2023
}
```

运行结果：

3/1/2022

3/29/2022

4/29/2022

5/2/2023

请按任意键继续...

102

```
#include <iostream>
using namespace std;
void main()
{ char s1[20], s2[20], s3[20];
  cin.getline(s1, 20); //省略了第3个输入结束符参数,缺省为 '\n'
  cin.get(s2, 20, '\n');
  cin.get(s3, 20); //省略了第3个输入结束符参数,缺省为 '\n'
  cout << s1 << s2 << s3;
}
```

运行结果:

abcd

efgh

abcdefgh请按任意键继续...

运行结果与原来的一样，可以看到getline和get都缺省了第3个输入结束符参数，自动缺省值为'\n'。

103

第2次作业:

见网络学堂第2次作业

2题必做，1题选做